

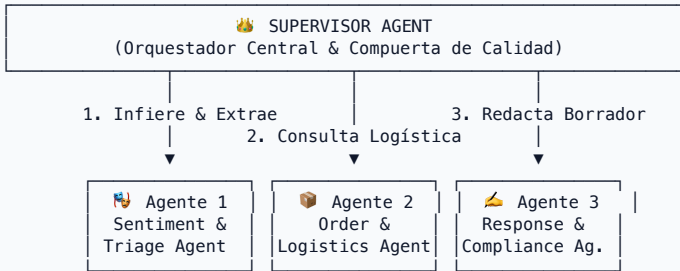
Propuesta Arquitectónica: Sistema Multi-Agente Jerárquico para Atención al Cliente en E-Commerce (Twitter/X)

Documento: Propuesta de Diseño Técnico Patrón: Supervisor Jerárquico Framework: LangGraph

Resumen Ejecutivo: Diseño técnico de un sistema multi-agente basado en una **topología jerárquica (Patrón Supervisor)** implementado sobre **LangGraph**. El sistema automatiza el triaje de quejas, la verificación del estado de pedidos en bases de datos y la redacción de respuestas personalizadas y seguras en Twitter/X, priorizando control determinista, trazabilidad y compuerta de calidad (*Quality Gate*).

1. Identificación y Definición de Roles

La arquitectura desacopla el razonamiento en un **nodo Supervisor central** y **tres agentes especialistas (Workers)**:



🔥 1.1. Supervisor Agent (Orquestador Central / Director de Tráfico)

Rol: Cerebro coordinador. No ejecuta tareas operativas directamente (evitando el anti-patrón de *Supervisor Todólogo*), sino que analiza el estado global, determina qué especialista interviene mediante `next_step` y realiza el control de calidad final (*Quality Gate*).

Modelo Recomendado: GPT-4o o Claude 3.5 Sonnet .

🗨️ 1.2. Sentiment & Triage Agent (Analista de Sentimiento y Extracción)

Rol: Especialista en NLP para clasificar emoción (positivo, neutro, negativo moderado, crítico), calcular urgencia (0.0 a 1.0) y extraer `#ORD-XXXXX` y `@usuario` .

Modelo Recomendado: GPT-4o-mini .

📦 1.3. Order & Logistics Agent (Especialista en Órdenes y Logística)

Rol: Especialista en integración transaccional para consultar la base de datos de órdenes y la API de mensajería (DHL/FedEx) para recuperar estado de despacho y motivos de retraso.

Modelo Recomendado: GPT-4o-mini .

🖋️ 1.4. Response & Compliance Agent (Redactor y Cumplimiento)

Rol: Especialista en tono empático de marca, límite estricto de 280 caracteres de Twitter/X y enmascaramiento de datos personales (PII).

Modelo Recomendado: GPT-4o-mini .

2. Definición de Herramientas Técnicas (Tooling Contract)

2.1. Herramientas del Sentiment & Triage Agent

- `classify_sentiment_and_urgency(tweet_text: str)` : Retorna `SentimentData(sentiment, urgency_score, category)` . Ante texto ambiguo o spam, asigna categoría `"DESCONOCIDO"` y urgencia `0.1` .
- `extract_entities_pydantic(tweet_text: str)` : Retorna `EntitiesData(order_id, customer_handle)` . Si no detecta ID, retorna `order_id = None` para solicitarlo por DM.

2.2. Herramientas del Order & Logistics Agent

- `query_order_db(order_id: str)` : Retorna `OrderRecord(status, tracking_id, customer_id, order_exists)` . Si no existe, retorna `order_exists = False` sin romper el flujo.

- `get_courier_tracking_info(tracking_id: str)` : Retorna `CourierStatus(carrier, location, eta, delay_reason)` . Ante caídas de API aplica *Degradación Grácil* con datos en caché.

2.3. Herramientas del Response & Compliance Agent

- `twitter_length_formatter(draft: str)` : Retorna `FormattedTweet(final_text, char_count, is_valid)` . Trunca respetando límites de palabras y agrega llamada a DM si excede 280 caracteres.
- `pii_sanitizer_filter(text: str)` : Retorna `CleanText(sanitized_text, pii_detected)` . Enmascara automáticamente tarjetas (`***-1234`), correos y documentos.

3. Esquema de Estado y Flujo Lógico

3.1. Esquema del Estado Global (CustomerSupportState)

```
from typing import Annotated, Literal, Optional, TypedDict
from langchain_core.messages import BaseMessage
from langgraph.graph.message import add_messages
from pydantic import BaseModel, Field

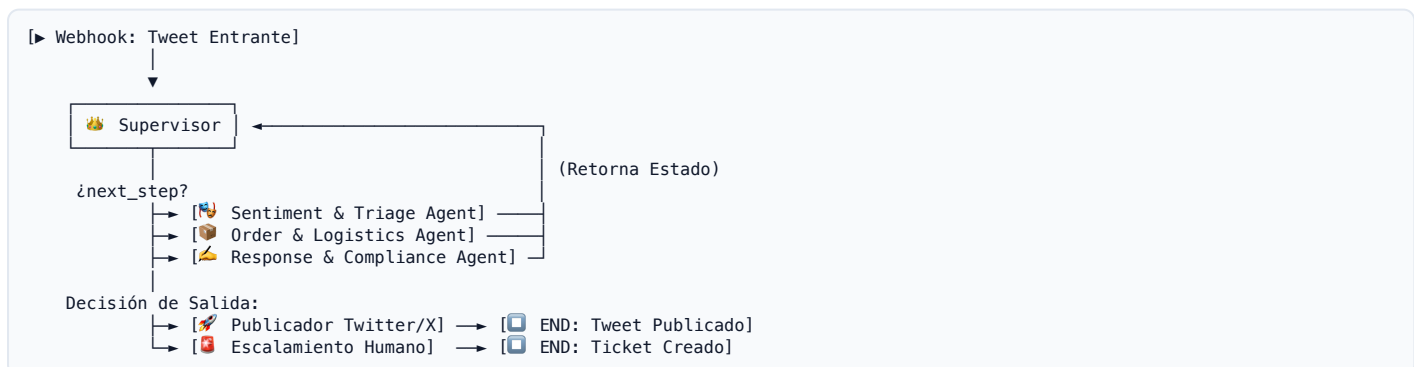
class SentimentPayload(BaseModel):
    sentiment: Literal["POSITIVO", "NEUTRO", "NEGATIVO_MODERADO", "NEGATIVO_CRÍTICO"]
    urgency_score: float = Field(ge=0.0, le=1.0)
    category: str
    order_id: Optional[str] = None
    customer_handle: str

class LogisticsPayload(BaseModel):
    order_exists: bool
    status: str
    tracking_url: Optional[str] = None
    delay_reason: Optional[str] = None
    estimated_delivery: Optional[str] = None

class ResponsePayload(BaseModel):
    tweet_text: str
    char_count: int
    invites_to_dm: bool
    requires_human_approval: bool

class CustomerSupportState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
    raw_tweet: str
    sentiment_data: Optional[SentimentPayload]
    logistics_data: Optional[LogisticsPayload]
    response_data: Optional[ResponsePayload]
    next_step: Literal["triage_agent", "logistics_agent", "response_agent", "publish_tweet", "human_escal"]
    iteration_count: int
    is_escalated: bool
```

3.2. Diagrama de Flujo del Grafo en LangGraph



3.3. Secuencia Paso a Paso de Transferencia de Estado

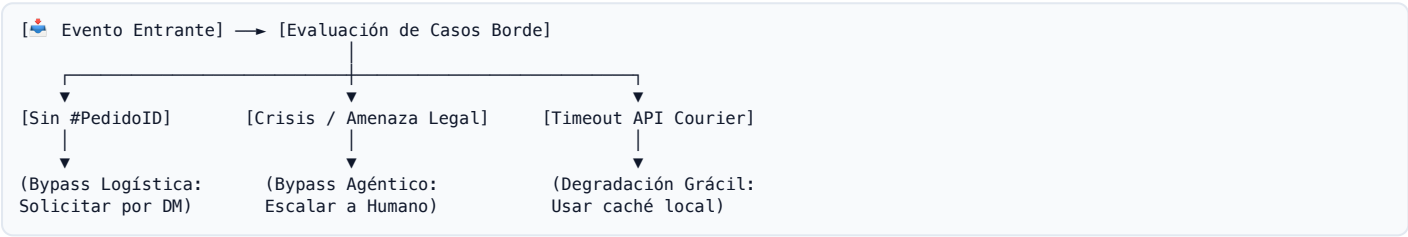
1. **Paso 1 (Ingesta):** Llega el tweet. El Supervisor inicializa el estado y enruta a `next_step = "triage_agent"` .
2. **Paso 2 (Triage):** Ejecuta análisis emocional. Mutación: `"sentiment_data": {"sentiment": "NEGATIVO_CRÍTICO", "urgency_score": 0.95, "order_id": "ORD-4491", ...}` .
3. **Paso 3 (Decisión Intermedia):** El Supervisor detecta ID de orden y enruta a `next_step = "logistics_agent"` .

- 4. **Paso 4 (Logística):** Consulta DB y courier. Mutación: `"logistics_data": {"order_exists": true, "status": "RETRASADO", "estimated_delivery": "Mañana 14:00hs", ...}`.
- 5. **Paso 5 (Redacción):** El redactor genera el mensaje. Mutación: `"response_data": {"tweet_text": "Hola @cliente_enojado...", "char_count": 168, ...}`.
- 6. **Paso 6 (Quality Gate & Publicación):** El Supervisor valida el cumplimiento y aprueba: `next_step = "publish_tweet" → END`.

4. Justificación Arquitectónica: Jerarquía vs Cooperación

- 1. **Mitigación de Riesgos de Marca:** En topología jerárquica el Supervisor es un *Quality Gate* centralizado obligatorio. En una red cooperativa cualquier agente podría publicar directamente exponiendo la marca a alucinaciones.
- 2. **Determinismo del Flujo:** El Supervisor impone un orden protocolar estricto (Triage → Logística → Redacción → Validación).
- 3. **Control de Bucles (Anti-Loop):** Previene ciclos infinitos mediante contadores de recursión centralizados.
- 4. **Eficiencia de Costos:** Modelos heterogéneos (`GPT-4o` en Supervisor y `GPT-4o-mini` en Workers) permiten ahorrar más del 70% en costos de inferencia.
- 5. **Mínimo Privilegio:** Aislamiento estricto de datos; el redactor no tiene acceso directo a la base de datos de pedidos.

5. Matriz de Casos Borde y Resiliencia (Edge Cases)



- Caso 1 (Sin ID de Pedido):** Se saltea el nodo logístico y el redactor emite un tweet solicitando el ID por DM.
- Caso 2 (Crisis de Marca / Legal):** `urgency_score = 1.0` activa bypass agéntico inmediato escalando a soporte humano prioritario.
- Caso 3 (Timeout API Courier):** Aplica degradación grácil recuperando la última fecha en caché de la DB interna.
- Caso 4 (Pedido Inexistente):** Emite respuesta amable informando que el código no coincide y ofrece canal de asistencia.

6. Observabilidad y Guardrails Operativos

- Anti-Loop:** Tope estricto de `recursion_limit = 4`. Si se supera, escala automáticamente a operador humano.
- Mitigación de Cuello de Botella:** Prompt Caching en Supervisor, salidas estructuradas y llamadas asíncronas.
- Observabilidad (LangSmith):** `Trace (Tweet ID) → Span (Supervisor) → Span (Worker) → Span (Tool Call)`.
- SLOs:** `TTFR ≤ 45s`, Aprobación de Quality Gate `≥ 90%`, Costo `≤ $0.003 USD` por interacción.

7. Conclusión

La **Topología Jerárquica con Patrón Supervisor** es la solución de ingeniería óptima para automatización en redes públicas, uniendo especialización técnica con riguroso control de calidad y mitigación de riesgos de marca.